IOCP 정리

Oct 17, 2026 · @성준

1. 들어가며

대규모 동시 접속을 처리하는 서버(게임 서버, 실시간 서비스, 대용량 API 게이트웨이 등)를 Windows에서 구현할 때 사실상 표준으로 쓰이는 것이 IOCP다. 이 글에서는 IOCP가 왜 필요한지부터 내부 동작 원리, 실전 구현 패턴, 흔히 겪는 함정, 그리고 실제 사용법과 예시까지 순서대로 정리한다.

2. 동기 I/O 모델의 한계

2.1 Thread-per-connection

가장 직관적인 서버 모델은 커넥션마다 스레드를 하나씩 붙이는 방식이다.

```
while (true) {
    SOCKET client = accept(listenSocket, ...);
    std::thread(HandleClient, client).detach();
}
```

구현은 단순하지만 두 가지 문제가 있다.

- **메모리**: 스레드마다 스택 공간을 예약한다. 예약(reserve)은 가상 주소 공간이고 실제로 커밋되는 메모리는 사용한 만큼만 늘어나지만, 예약 크기만큼의 주소 공간은 차지한다. 기본 예약 크기는 실행 파일 헤더(링커의 `/STACK` 옵션)나 `CreateThread`의 `dwStackSize`로 정해지며, MSVC 링커의 기본값은 1MB다(MinGW로 빌드한 실행 파일은 헤더에 예약 2MB, 초기 커밋 4KB로 기록되는 경우가 있다). 1MB 기준으로 커넥션이 1만 개면 약 10GB의 주소 공간이 예약되어 32비트 프로세스에서는 불가능하고, 64비트에서도 스레드마다 커널 스택과 TEB 같은 고정 비용이 든다.
- **컨텍스트 스위칭 비용**: OS 스케줄러가 수천~수만 개의 스레드를 오가며 스위칭하는 비용이 실제 작업량보다 커진다.

2.2 select / WSAPoll 기반 이벤트 루프 (Reactor 패턴)

소켓 집합을 감시하다가 "읽기/쓰기가 가능한 상태"가 된 소켓만 골라 처리하는 방식이다. 스레드 수는 줄일 수 있지만, 매 호출마다 감시 대상 전체를 스캔하기 때문에 소켓 수가 늘수록 호출마다 처리할 대상이 늘어 오버헤드가 커진다(일반적으로 알려진 특성이다). 문서가 명시하는 한계로, `select`는 `fd_set`에 담을 수 있는 소켓 수가 `FD_SETSIZE`(기본 64)로 제한된다. 그리고 "가능하다"는 신호일 뿐 실제 읽기/쓰기는 애플리케이션이 수행해야 한다.

2.3 Proactor 패턴으로의 전환

IOCP는 이 둘과 근본적으로 다른 접근을 취한다. "지금 읽어도 된다"가 아니라 **"읽기가 이미 끝났다"**는 사실을 커널이 알려준다. 애플리케이션은 미리 비동기 I/O 요청(overlapped I/O)을 커널에 던져두고, 실제 데이터 전송은 커널이 백그라운드에서 수행한 뒤 완료만 통지받는다. 이것이 Proactor 패턴이며, IOCP는 Windows 커널이 이 패턴을 위해 제공하는 오브젝트다.

3. IOCP의 핵심 구조

3.1 완료 포트라는 커널 오브젝트

```
HANDLE hIOCP = CreateIoCompletionPort(
    INVALID_HANDLE_VALUE, // 새 포트를 생성
    NULL,                 // 기존 포트 없음
    0,                    // CompletionKey (신규 생성 시 미사용)
    0                     // 동시성 값 0 → 프로세서 수만큼 자동 설정
);
```

이렇게 만든 포트에 소켓 핸들을 연결한다.

```
CreateIoCompletionPort(
    (HANDLE)clientSocket,
    hIOCP,
    (ULONG_PTR)pSession,  // CompletionKey: 세션 포인터를 그대로 사용
    0
);
```

한 번 연결된 소켓은 이후 그 소켓에 대한 모든 비동기 I/O의 완료가 이 포트의 **내부 완료 큐**에 자동으로 쌓인다. 패킷은 FIFO 순서로 큐에 들어가지만, 꺼내는 순서와 처리가 끝나는 순서까지 FIFO가 보장되는 것은 아니다(아래 참고). CompletionKey는 임의의 값을 넣을 수 있는데, 보통 세션 객체를 식별할 수 있는 포인터나 ID를 넣어 완료 시 어떤 세션의 I/O인지 바로 찾을 수 있게 한다.

큐에 들어가는 순서와 처리되는 순서는 다르다. 완료 패킷은 FIFO로 큐에 들어가며 일반적으로 오래된 것부터 꺼내지지만, Microsoft 문서는 다른 순서로 꺼내질 수 있다고 명시한다. 또 포트에서 대기 중인 워커는 LIFO 순서로 깨워진다(가장 최근에 대기한 스레드가 가장 오래된 완료를 받는다). 또 여러 워커가 서로 다른 패킷을 동시에 처리하므로, 먼저 꺼낸 패킷의 처리가 먼저 끝난다는 보장이 없다. 같은 세션의 Recv 완료와 Send 완료, 심지어 연속된 두 완료도 서로 다른 워커에서 동시에 처리될 수 있다는 뜻이다. 그래서 세션 상태는 락 등으로 보호하고, 한 소켓에 Recv는 하나만 걸어 수신 순서가 뒤바뀌지 않게 한다(7.1절).

3.2 OVERLAPPED 구조체 — 요청과 완료를 잇는 열쇠

비동기 I/O를 걸 때마다 `OVERLAPPED` 구조체(혹은 이를 포함하는 확장 구조체)를 함께 넘긴다.

```
struct IoContext : OVERLAPPED {
    WSABUF wsaBuf;
    IoType type;  // Recv, Send 등 사용자 정의
    // ...
};
```

I/O가 완료되면 커널은 이 포인터를 그대로 완료 큐에 실어 보낸다. 즉 `OVERLAPPED`를 상속/포함한 구조체를 만들어두면, 완료 시 `static_cast`로 되돌려서 "이게 무슨 요청이었는지" 알아낼 수 있다. 가상 함수나 다중 상속이 있으면 `OVERLAPPED` 서브객체의 주소가 객체 시작 주소와 다를 수 있으므로, 커널에 넘길 때와 되돌릴 때 모두 `reinterpret_cast`가 아니라 `static_cast`를 쓴다. 이는 하나의 소켓에 여러 개의 I/O(예: Recv와 Send를 동시에)를 걸어둘 수 있게 해주는 핵심 메커니즘이다. 동시에 진행 중인 I/O마다 `OVERLAPPED`는 별개여야 하고, 완료 통지를 받을 때까지 유효해야 한다.

`OVERLAPPED.hEvent`에 유효한 이벤트 핸들을 넣고 그 값의 최하위 비트를 1로 세팅하면 완료 패킷이 완료 포트로 큐잉되지 않는다(문서 `GetQueuedCompletionStatus` / `CreateIoCompletionPort`의 비고). 이 글처럼 완료 포트로 통지받는 경우에는 `hEvent`를 쓰지 않고 0으로 둔다.

3.3 비동기 요청 걸기 — WSARecv / WSASend

```
DWORD flags = 0;
int ret = WSARecv(
    socket,
    &pContext->wsaBuf, 1,
    NULL,                  // 즉시 완료 시 바이트 수(overlapped라 보통 NULL)
    &flags,
    static_cast<OVERLAPPED*>(pContext),
    NULL
);

if (ret == SOCKET_ERROR) {
    int err = WSAGetLastError();
    if (err != WSA_IO_PENDING) {
        // 진짜 에러 — 연결 종료 처리
    }
    // WSA_IO_PENDING은 정상: 비동기로 진행 중이라는 뜻
}
```

`WSA_IO_PENDING`은 에러가 아니라 "커널이 백그라운드에서 처리 중"이라는 정상 신호다. 이 구분을 놓치면 정상적으로 동작 중인 I/O를 에러로 오판하는 버그가 흔히 발생한다.

3.4 완료 수신 — GetQueuedCompletionStatus

워커 스레드들은 이 함수 하나로 대기하며 완료된 I/O를 받아간다.

```
BOOL GetQueuedCompletionStatus(
    HANDLE hCompletionPort,
    LPDWORD lpNumberOfBytesTransferred,
    PULONG_PTR lpCompletionKey,
    LPOVERLAPPED* lpOverlapped,
    DWORD dwMilliseconds
);
```

전형적인 워커 루프:

```
void WorkerLoop(HANDLE hIOCP) {
    for (;;) {
        DWORD bytesTransferred = 0;
        ULONG_PTR completionKey = 0;
        OVERLAPPED* pOverlapped = nullptr;

        BOOL ok = GetQueuedCompletionStatus(
            hIOCP, &bytesTransferred, &completionKey, &pOverlapped, INFINITE
        );

        if (pOverlapped == nullptr) {
            // I/O 완료가 아니다: 종료 신호(PostQueuedCompletionStatus)이거나 큐 대
기 자체의 실패
            if (ok && completionKey == KEY_SHUTDOWN) break;
            if (!ok) break;   // INFINITE 대기에서의 실패는 포트 핸들 문제다. 재시도하
면 CPU를 태우며 공회전한다.
            continue;
        }

        auto* pSession = reinterpret_cast<Session*>(completionKey);
        auto* pContext = static_cast<IoContext*>(pOverlapped);

        if (!ok) {
            // I/O가 에러로 완료됨(연결 리셋, 취소 등). GetLastError()로 원인을 구분한
다
            HandleDisconnect(pSession, GetLastError());
            continue;
        }

        if (bytesTransferred == 0 && pContext->type == IoType::Recv) {
            // Recv가 0바이트로 성공: 상대가 정상 종료(graceful close)했다
            HandleDisconnect(pSession, ERROR_SUCCESS);
            continue;
        }

        switch (pContext->type) {
```

```
            case IoType::Recv: HandleRecvCompleted(pSession, pContext,
bytesTransferred); break;
            case IoType::Send: HandleSendCompleted(pSession, pContext,
bytesTransferred); break;
        }
    }
}
```

이 예시는 개념을 보이기 위해 `CompletionKey`에 세션 포인터를 그대로 넣었다. 세션이 완료 통지보다 먼저 파괴되면 이 포인터는 허상이 되므로, 실제 코드는 4장의 규칙에 따라 컨텍스트가 세션의 `shared_ptr`를 쥐게 하고 키는 식별용으로만 쓴다(7.2절 예제도 컨텍스트가 세션의 `shared_ptr`를 쥐고 키는 읽지 않는다).

`pOverlapped`가 `NULL`인 경우를 가장 먼저 걸러야 한다. 이 경우 `pContext`는 유효한 객체가 아니므로 `pContext->type`을 읽으면 안 된다.

주기 작업을 위해 `INFINITE` 대신 유한 타임아웃을 쓰는 워커는 `NULL` 분기를 이렇게 구분한다.

```
if (pOverlapped == nullptr) {
    if (ok) {                                   // PostQueuedCompletionStatus
로 넣은 사용자 신호
        if (completionKey == KEY_SHUTDOWN) break;
        continue;
    }
    DWORD err = GetLastError();
    if (err == WAIT_TIMEOUT) { OnIdle(); continue; }   // 시간 만료: 정상
    break;                                       // ERROR_ABANDONED_WAIT_0 등:
포트가 닫혔다. 재시도하지 않는다
}
```

여기서 반드시 구분해야 할 세 가지 케이스가 있다.

- `ok == FALSE`이고 `pOverlapped == NULL`: 완료 패킷을 꺼내지 못한 것이다. `GetLastError()`로 구분한다. 유한 타임아웃에서는 `WAIT_TIMEOUT`(시간 만료)이 정상적인 경우이고(문서는 타임아웃을 "`FALSE`를 반환하고 `*lpOverlapped`를 `NULL`로 설정한다"고만 서술하며, `GetLastError()`의 `WAIT_TIMEOUT`은 실무에서 널리 쓰는 값이다), 그 밖의 값(대기 중 포트 핸들이 닫힌 경우의 `ERROR_ABANDONED_WAIT_0` 등)이 실제 오류다. 이때 바이트 수와 키 출력값은 불확정이므로 읽지 않는다. `INFINITE` 대기에는 타임아웃이 없으므로 이 경우가 모두 오류이며, 재시도하지 않고 워커를 종료한다
- `ok == FALSE`이고 `pOverlapped != NULL`: I/O 자체는 완료됐지만 에러 상태(연결 리셋 등) — `GetLastError()`로 원인 확인
- `ok == TRUE`이고 `bytesTransferred == 0`: 상대가 정상적으로 연결을 종료(graceful close)했다는 뜻(Recv 기준)

이 세 케이스를 뭉뚱그려 처리하면 정상 종료와 비정상 종료를 구분하지 못하는 버그로 이어진다.

3.5 동시성 값(NumberOfConcurrentThreads)의 의미

`CreateIoCompletionPort`의 네 번째 인자는 흔히 오해하는 부분이다. 이는 "워커 스레드를 몇 개 만들어야 하는가"가 아니라, **"이 포트와 연결된 스레드 중 동시에 실행(러너블) 상태일 수 있는 수의 상한"**이다. 스레드는 `GetQueuedCompletionStatus`를 처음 호출할 때 그 포트와 연결되며(한 스레드는 한 포트에만 연결된다), 이 값은 그렇게 연결된 워커 스레드의 동시 실행을 제어한다. 값이 0이면 시스템의 프로세서 수를 기본값으로 사용한다.

- 러너블 스레드 수가 이 값에 도달하면, 이후 완료를 받으려는 스레드는 실행 중인 스레드가 줄어들 때까지 대기한다. 처리 중인 스레드가 블로킹 호출 등으로 대기 상태에 들어가면 러너블 수가 줄어, 대기하던 다른 워커가 깨어나 완료를 처리할 수 있다.
- 그래서 워커 스레드는 이 값보다 많이 만들어 둔다. Microsoft 문서의 경험칙은 프로세서 수의 최소 2배다. 이는 출발점으로 쓰는 경험칙이지 필수 조건이 아니며, 최적값은 워크로드에 따라 측정해서 정한다.
- 완료 핸들러가 동기 디스크 I/O, Redis 같은 외부 호출, 긴 계산을 하면 그 워커는 그동안 다른 완료를 처리하지 못한다. 동시성 값 메커니즘은 막힌 스레드 대신 대기 중인 다른 워커를 깨우지만, 워커가 모두 막히면 처리량이 떨어지고 완료가 쌓인다. 이런 작업은 별도 작업 큐(스레드 풀)로 넘기고 IOCP 워커는 I/O 완료 처리와 짧은 상태 갱신에만 쓴다. 문서도 긴 계산이 필요한 경우에는 더 큰 동시성 값이 더 많은 패킷을 동시에 처리하게 해 준다고 설명한다.
- 블로킹에서 돌아오는 스레드 때문에 러너블 스레드 수가 일시적으로 이 값을 넘을 수 있다. 이 값은 엄격한 CPU 점유 상한이 아니라 스케줄링을 조절하는 장치다.
- 8코어 머신에서 값을 8로 주면, 워커를 16개 만들어도 완료를 처리 중인 러너블 스레드는 대체로 8개 안팎으로 유지되어 불필요한 컨텍스트 스위칭과 캐시 스레싱이 줄어든다. 완료 핸들러가 오래 계산하는 작업이라면 더 큰 값이 유리할 수 있으므로 프로파일링으로 정한다.
- 직접 스레드를 관리할 필요가 없는 새 코드는 IOCP를 내부적으로 사용하는 스레드 풀 API(`CreateThreadpoolIo`)도 고려한다.

3.6 AcceptEx/ConnectEx, Zero-byte Recv, 소켓 재사용 등 실전 기법

- **AcceptEx / ConnectEx**: 이들도 overlapped I/O로 걸 수 있어, accept·connect 자체도 IOCP 완료 큐를 통해 비동기로 처리할 수 있다. `accept()`를 블로킹으로 별도 스레드에서 도는 구조보다 일관성이 높다.
- **Zero-byte Recv**: 버퍼 크기 0으로 `WSARecv`를 걸어 "데이터가 도착했다"는 신호만 받고, 실제 읽기는 완료 후 별도로 수행하는 기법. 큰 수신 버퍼를 세션마다 상시 점유하지 않아도 되는 장점이 있지만, 실제 읽기를 한 번 더 호출해야 하므로 지연이 늘어나는 트레이드오프가 있다.
- **소켓 재사용(DisconnectEx)**: `DisconnectEx`에 `TF_REUSE_SOCKET`을 주면 연결을 끊은 뒤 그 소켓 핸들을 `AcceptEx`나 `ConnectEx`에 다시 넘길 수 있다(문서가 허용하는 재사용 경로는 이 둘이다). 어떤 소켓을 어느 함수에 다시 넘길 수 있는지는 소켓이 어떻게 만들어지고 끊겼는지에 따라 달라지며 `AcceptEx`와 `ConnectEx` 각 문서의 비고에 따로 적혀 있으므로, 재사용 경로를 구현하기

전에 두 문서를 모두 확인한다. 소켓 생성 비용은 줄지만, TCP의 `TIME_WAIT` 때문에 `DisconnectEx`가 지연될 수 있고 연결되지 않은 소켓에는 `WSAENOTCONN`으로 실패한다. 이 글의 예제는 소켓 재사용을 구현하지 않는다.

4. 세션(커넥션) 수명 관리 — 가장 많이 틀리는 부분

IOCP 자체는 매우 안정적인 API지만, 실무에서 버그의 8할은 "세션이 언제 안전하게 파괴되어도 되는가"에서 나온다.

4.1 문제의 본질

소켓을 `closesocket()`으로 닫아도, 이미 커널에 제출되어 진행 중이던 I/O의 완료 통지는 큐에 남아 나중에 도착할 수 있다. 세션 객체를 즉시 `delete`하면, 뒤늦게 도착한 완료 통지가 이미 해제된 메모리(`OVERLAPPED`를 포함한 컨텍스트, 세션 자체)를 참조하는 use-after-free가 발생한다.

4.2 일반적인 해법 — 참조 카운팅

- 세션에 진행 중인 I/O 개수를 원자적 카운터로 추적한다(Recv를 걸 때 +1, 완료 시 -1 등).
- `close` 요청이 오면 소켓은 즉시 닫되(더 이상 새 I/O를 받지 않도록), 실제 세션 객체 파괴는 "카운터가 0이 될 때"까지 미룬다.
- `shared_ptr` / `enable_shared_from_this` 조합으로 I/O 컨텍스트가 세션의 소유권을 들고 있게 하면, 완료 통지가 늦게 와도 세션이 먼저 파괴되지 않는다. 이 방식은 위의 카운터 방식과 같은 목적을 이루므로 둘 중 하나로 충분하다. 7.2절 예제는 `shared_ptr` 방식만 쓴다.

```
class Session : public std::enable_shared_from_this<Session> {
    std::atomic<int> _outstandingIo{0};
public:
    void PostRecv() {
        _outstandingIo.fetch_add(1, std::memory_order_relaxed);
        auto self = shared_from_this(); // 컨텍스트가 세션을 붙잡음
        // WSARecv 호출...
    }
    void OnIoCompleted() {
        if (_outstandingIo.fetch_sub(1, std::memory_order_acq_rel) == 1) {
            // 마지막 I/O였다면 최종 정리(FinalizeClose 등) 수행
        }
    }
};
```

이 원칙("진행 중인 I/O가 0이 될 때까지 실제 자원 해제를 미룬다")은 IOCP 기반 서버 설계에서 가장 중요한 불변식(invariant) 중 하나다.

4.3 I/O 컨텍스트 수명 규칙 (불변식)

모든 I/O 종류(Accept, Connect, Recv, Send)에 같은 규칙을 적용한다.

1. 발행에 성공한 요청(반환 `0` 또는 `WSA_IO_PENDING`)은 정확히 하나의 완료 통지를 만든다. 반환 `0`(즉시 완료)이어도 `FILE_SKIP_COMPLETION_PORT_ON_SUCCESS`를 설정하지 않았다면 평소처럼 통지가 큐에 들어오므로, 그 자리에서 따로 처리하거나 회수하지 않는다.
   (`SetFileCompletionNotificationModes` 문서는 이 플래그를 "평소라면 큐에 넣을 완료 항목을 큐에 넣지 않는" 옵션으로 설명하고, 소켓에서는 IFS 핸들을 쓰는 LSP와만 호환된다고 밝힌다. 이 글은 플래그를 쓰지 않는다.)
2. 그 완료 통지를 처리하는 워커가 컨텍스트의 유일한 회수 주체다. 발행이 즉시 실패한 요청은 통지가 오지 않으므로 발행한 쪽이 회수한다. 두 경로가 겹치는 일은 없다.
3. Recv/Send는 호출 전에 컨텍스트에 세션의 `shared_ptr`(`owner`)를 걸고, 즉시 실패하면 되돌린다. 완료 통지가 도착하는 동안 세션은 파괴되지 않는다.
4. 취소는 `closesocket()`(또는 `CancelIoEx`)로 한다. 취소된 I/O도 에러 완료(`WSA_OPERATION_ABORTED`, SDK에서 `ERROR_OPERATION_ABORTED`와 같은 값)를 만든다. `closesocket`은 취소를 시작할 뿐 반환 시점에 완료가 끝났다는 보장이 없고, 문서도 완료되기 전에는 `OVERLAPPED` 같은 자원을 정리하지 말라고 안내한다. 따라서 취소는 회수를 대체하지 않는다.
5. 세션은 모든 컨텍스트의 `owner`가 풀린 뒤에만 파괴된다. 서버는 모든 세션이 소멸하고 AcceptEx/ConnectEx 컨텍스트가 회수된 뒤에만 워커를 종료한다.

종류	진행 중 소유자	완료 통지 처리	발행 즉시 실패	취소
Accept	진행 중 Accept 카운터	재제출한 뒤 `unique_ptr`로 회수하고 카운트 감소	발행 함수가 실패를 알리면 `unique_ptr`가 해제하고 카운트 감소	리슨 소켓 `closesocket` → 에러 완료로 회수
Connect	진행 중 Connect 카운터	소켓을 세션으로 넘기거나 닫고, `unique_ptr` 회수 후 카운트 감소	발행한 쪽이 소켓을 닫고 카운트 감소	취소하지 않고 완료(OS 타임아웃 포함)를 기다림
Recv	대기 중인 수신 컨텍스트와 그 `owner`	처리 후 다음 Recv 발행(송신 큐가 high-water면 보류)	`unique_ptr`가 컨텍스트와 `owner`를 해제하고 세션 `Close`	세션 `Close`
Send	송신 컨텍스트와 그 `owner`	부분 전송이면 같은 컨텍스트로 재발행, 아니면 큐에서 제거하고 다음 Send 발행	`unique_ptr`가 해제하고 세션 `Close`	세션 `Close`

이 불변식은 정상 경로 테스트만으로는 검증되지 않는다. 생성된 컨텍스트 수를 세는 카운터(`liveCount`)를 두고 서버 종료 시 확인하면 회수 누락(양수)과 이중 회수(음수)를 잡을 수 있다. 다만 `liveCount == 0`은 유용한 지표일 뿐 메모리 안전성을 증명하지는 않는다. 누락과 이중 회수가 서로 상쇄되어 0이 될 수 있고, 해제된 메모리를 참조하는 오류는 카운터로 드러나지 않는다. 그래서 종료 시 다음을 함께 확인한다.

- 진행 중인 Accept와 Connect 카운터가 0이다.
- 모든 세션이 소멸했다(세션 카운터 0).
- 모든 Recv와 Send 완료가 처리되었다. 세션이 소멸했다면 각 컨텍스트의 `owner`가 모두 풀린 것이므로 이 조건을 간접적으로 보장한다.
- 컨텍스트 회수 누락이나 이중 회수가 없다(`liveCount`가 정확히 0이고 시험 중 음수가 된 적이 없다).
- 모든 워커가 종료되었다(`join` 완료).

해제된 메모리의 접근 같은 오류는 PageHeap, AddressSanitizer 같은 도구와 시나리오 반복으로 따로 확인한다.

5. 성능 관점에서 본 IOCP

- **스레드 수가 커넥션 수에 비례하지 않는다**: 워커 스레드 수는 프로세서 수를 기준으로 정하고(3.5절), 커넥션 수와는 무관하다. 커넥션은 수만 개 이상까지 확장할 수 있으며 실제 한계는 메모리, 핸들, 네트워크 대역폭과 핸들러의 작업량이 정한다.
- **락 경합 최소화**: IOCP 자체의 큐잉과 스케줄링은 커널이 담당하므로 이벤트 루프를 위한 별도의 락이 필요 없다. 다만 여러 워커가 같은 세션의 완료를 동시에 처리할 수 있으므로(3.1절), 세션 상태를 보호하는 락은 애플리케이션이 따로 둔다. 락 구간을 짧게 하고 소켓 호출 외의 무거운 작업은 락 밖에서 한다.
- **버퍼 관리가 성능에 큰 영향**: overlapped I/O 도중에는 버퍼가 유효해야 하므로, 세션마다 버퍼를 고정 할당하거나 오브젝트 풀에서 재사용하는 방식이 일반적이다. 매 I/O마다 `new` / `delete`로 버퍼를 할당하면 IOCP의 이점을 스레드 관리에서만 얻고 메모리 할당에서 잃게 된다.

6. API 레퍼런스 — 주요 함수 인터페이스

6.1 CreateIoCompletionPort

완료 포트를 새로 만들거나, 기존 포트에 파일/소켓 핸들을 연결한다. 두 역할을 하나의 함수가 겸한다.

```
HANDLE CreateIoCompletionPort(
    HANDLE    FileHandle,
    HANDLE    ExistingCompletionPort,
    ULONG_PTR CompletionKey,
    DWORD     NumberOfConcurrentThreads
);
```

파라미터	설명
`FileHandle`	포트에 연결할 핸들. 소켓, 파일, 파이프 등 overlapped I/O를 지원하는 핸들이면 된다. **새 포트만 생성**할 때는 `INVALID_HANDLE_VALUE`를 넘긴다.
`ExistingCompletionPort`	기존 포트 핸들. **새 포트 생성** 시에는 `NULL`. 기존 소켓을 특정 포트에 **연결**할 때는 그 포트의 핸들을 넘긴다.
`CompletionKey`	이 핸들에 대한 완료 통지마다 함께 전달될 사용자 정의 값. 보통 세션/커넥션을 가리키는 포인터를 캐스팅해서 넣는다. 새 포트 생성 시에는 무시된다.
`NumberOfConcurrentThreads`	포트와 연결된 스레드 중 동시에 실행(러너블) 가능한 수의 상한(3.5절). `0`이면 시스템의 프로세서 개수를 기본값으로 사용. **새 포트 생성** 시에만 의미가 있고, 기존 소켓 연결 호출에서는 무시된다.

반환값: 성공 시 완료 포트 핸들(신규 생성이든 연결이든 동일 핸들 반환), 실패 시 `NULL`(`GetLastError()`로 원인 확인).

하나의 핸들은 하나의 완료 포트에만 연결할 수 있고, 연결한 뒤에는 핸들을 닫을 때까지 유지된다. 여러 핸들이 같은 포트를 공유하는 것은 가능하다. 연결할 핸들은 overlapped I/O용으로 열려 있어야 한다(소켓은 `socket()`으로 만들면 기본이 overlapped이고, `WSASocket`에서는 `WSA_FLAG_OVERLAPPED`로 명시한다). 포트 자체는 모든 핸들 참조를 정리한 뒤 `CloseHandle`로 닫는다.

공식 문서의 추가 주의점은 다음과 같다. 완료 포트 핸들은 스레드 사이에서는 공유할 수 있지만 프로세스 사이에서는 공유할 수 없다. 완료 키는 핸들마다 고유하게 정해 어느 핸들의 완료인지 구별할 수 있게 한다. 포트에 연결된 핸들에는 `ReadFileEx`/`WriteFileEx`를 쓸 수 없다. 연결된 핸들을 상속하거나 `DuplicateHandle`로 복제하는 것은 피한다(복제본의 I/O도 같은 포트로 완료 통지를 만든다). 공식 문서의 `CreateIoCompletionPort` 페이지에는 예제 코드가 없고 문법만 있다.

사용 패턴은 항상 두 단계다.

```
// ① 포트 생성
HANDLE hIOCP = CreateIoCompletionPort(INVALID_HANDLE_VALUE, NULL, 0, 0);
// ② 소켓을 그 포트에 연결
CreateIoCompletionPort((HANDLE)clientSocket, hIOCP, (ULONG_PTR)pSession, 0);
```

6.2 GetQueuedCompletionStatus

워커 스레드가 완료된 I/O를 하나 받아올 때 호출한다. 이 스레드에 완료 항목이 없으면 지정한 시간만큼 블로킹한다.

```
BOOL GetQueuedCompletionStatus(
    HANDLE       CompletionPort,
    LPDWORD      lpNumberOfBytesTransferred,
    PULONG_PTR   lpCompletionKey,
    LPOVERLAPPED *lpOverlapped,
    DWORD        dwMilliseconds
);
```

파라미터	설명
`CompletionPort`	대기할 완료 포트 핸들
`lpNumberOfBytesTransferred`	[출력] 완료된 I/O에서 실제로 전송된 바이트 수
`lpCompletionKey`	[출력] 해당 핸들을 포트에 연결할 때 지정했던 `CompletionKey` — 보통 세션 식별용
`lpOverlapped`	[출력] 완료된 I/O에 사용됐던 `OVERLAPPED` 구조체(또는 이를 포함한 확장 구조체)의 포인터 — 어떤 요청이었는지 식별하는 열쇠
`dwMilliseconds`	대기 시간(ms). `INFINITE`면 무한 대기, `0`이면 즉시 반환(폴링)

반환값과 판별 방법:

반환값	`lpOverlapped`	의미
`TRUE`	유효	I/O 정상 완료. `bytesTransferred == 0`이면(Recv 기준) 상대가 정상 종료(graceful close)한 것
`FALSE`	`NULL`	완료 패킷을 꺼내지 못했다. `GetLastError()`로 구분한다: `WAIT_TIMEOUT`(유한 타임아웃 만료, 정상), `ERROR_ABANDONED_WAIT_0`(대기 중 포트 핸들이 닫힘) 등. 바이트 수와 키 출력값은 불확정이므로 읽지 않는다

반환값	`lpOverlapped`	의미
`FALSE`	유효	I/O는 완료됐으나 에러 상태로 완료됨(연결 리셋 등) — `GetLastError()`로 원인 확인

이 세 가지 조합을 정확히 구분해서 처리하는 것이 IOCP 서버 안정성의 기본이다.

6.3 GetQueuedCompletionStatusEx

여러 완료 항목을 한 번에 배치로 받아올 수 있는 확장 버전(Vista/Server 2008+).

```
BOOL GetQueuedCompletionStatusEx(
    HANDLE             CompletionPort,
    LPOVERLAPPED_ENTRY lpCompletionPortEntries,
    ULONG              ulCount,
    PULONG             ulNumEntriesRemoved,
    DWORD              dwMilliseconds,
    BOOL               fAlertable
);
```

파라미터	설명
`CompletionPort`	대기할 완료 포트 핸들
`lpCompletionPortEntries`	[출력] `OVERLAPPED_ENTRY` 배열. 각 원소가 `lpOverlapped` / `lpCompletionKey` / `dwNumberOfBytesTransferred`를 담고 있다
`ulCount`	배열의 크기(한 번에 받을 수 있는 최대 항목 수)
`ulNumEntriesRemoved`	[출력] 실제로 받아온 항목 수
`dwMilliseconds`	대기 시간(ms)
`fAlertable`	`TRUE`면 alertable wait 상태로 대기(APC 큐 처리 가능). 보통 `FALSE`

한 번의 호출로 여러 완료를 받을 수 있어 `GetQueuedCompletionStatus()`를 반복 호출하는 것보다 호출 횟수를 줄일 수 있다. 다만 항목마다 성공 여부를 따로 판별해야 하고 종료 신호도 배치 단위로 다뤄야 하므로 워커 로직이 복잡해진다. 처리량이 매우 중요한 경우가 아니라면 단건 버전으로도 충분한 경우가 많다.

6.4 PostQueuedCompletionStatus

실제 I/O 완료가 아니라, 애플리케이션이 임의로 완료 큐에 이벤트를 넣고 싶을 때 사용한다. 워커 스레드에 "종료해라" 같은 커스텀 신호를 보낼 때 흔히 쓴다.

```
BOOL PostQueuedCompletionStatus(
    HANDLE       CompletionPort,
    DWORD        dwNumberOfBytesTransferred,
    ULONG_PTR    dwCompletionKey,
    LPOVERLAPPED lpOverlapped
);
```

파라미터	설명
`CompletionPort`	이벤트를 넣을 완료 포트
`dwNumberOfBytesTransferred`	`GetQueuedCompletionStatus()`가 그대로 돌려줄 값 — 의미는 애플리케이션이 정의
`dwCompletionKey`	마찬가지로 애플리케이션이 정의하는 값. 예: 특수 값으로 "종료 신호"를 표시
`lpOverlapped`	마찬가지. `NULL`을 넣고 `CompletionKey`만으로 종료 신호를 구분하는 방식이 흔함. 문서는 이 세 값을 시스템이 사용하거나 검증하지 않는다고 하며, `lpOverlapped`가 실제 `OVERLAPPED`를 가리킬 필요도 없다고 설명한다

`GetQueuedCompletionStatus`, `GetQueuedCompletionStatusEx`, `PostQueuedCompletionStatus`의 Microsoft Learn 페이지에는 예제 코드가 없고 문법과 설명만 있다. 이 글에서는 7.2절의 에코 서버가 이 세 함수의 사용 예 역할을 한다.

워커 스레드 개수만큼 이 함수를 호출해 각 워커가 "종료 신호"를 하나씩 받고 루프를 빠져나가게 하는 것이 일반적인 종료 패턴이다. 다만 신호를 넣는 것만으로 안전한 종료가 보장되지는 않는다. 신호는 이미 큐에 있는 완료 뒤에 줄을 서지만, 신호가 들어간 뒤에도 이미 발행된 I/O는 계속 완료되고 완료 핸들러가 새 I/O를 발행할 수도 있다. 모든 워커가 빠져나간 뒤에 도착한 완료는 처리할 주체가 없어 컨텍스트와 세션이 회수되지 않는다. 진행 중인 모든 I/O를 먼저 회수하고 그 뒤에 신호를 보낸다(4.3절, 7.2절).

6.5 WSARecv / WSASend

overlapped 모드로 비동기 수신/송신을 요청한다. 시그니처가 거의 동일하다.

```
int WSARecv(
    SOCKET                             s,
    LPWSABUF                           lpBuffers,
    DWORD                              dwBufferCount,
    LPDWORD                            lpNumberOfBytesRecvd,
    LPDWORD                            lpFlags,
    LPWSAOVERLAPPED                    lpOverlapped,
    LPWSAOVERLAPPED_COMPLETION_ROUTINE lpCompletionRoutine
);

int WSASend(
    SOCKET                             s,
    LPWSABUF                           lpBuffers,
    DWORD                              dwBufferCount,
    LPDWORD                            lpNumberOfBytesSent,
    DWORD                              dwFlags,
    LPWSAOVERLAPPED                    lpOverlapped,
    LPWSAOVERLAPPED_COMPLETION_ROUTINE lpCompletionRoutine
);
```

파라미터	설명
`s`	대상 소켓
`lpBuffers`	`WSABUF{ ULONG len; CHAR* buf; }` 배열. 여러 버퍼를 한 번에 넘길 수 있는 scatter/gather I/O 지원
`dwBufferCount`	`lpBuffers` 배열의 원소 수
`lpNumberOfBytesRecvd` / `lpNumberOfBytesSent`	즉시 완료된 경우 바이트 수가 채워진다. 문서는 `lpOverlapped` 가 `NULL` 이 아니면 이 인자에 **`NULL` 을 넘기라고** 권고한다(오류 방지). 바이트 수는 항상 완료 통지(`GetQueuedCompletionStatus()` 의 출력 값)로 받는다
`lpFlags` (Recv)	[입출력] `MSG_PEEK` 등 수신 플래그
`dwFlags` (Send)	송신 플래그(보통 `0`)
`lpOverlapped`	IOCP에 연결된 소켓이라면 반드시 유효한 `OVERLAPPED` 포인터를 넘겨야 비동기로 동작함
`lpCompletionRoutine`	APC 기반 콜백. **IOCP로 통지받으려면 `NULL` **을 넘긴다. 완료 루틴(APC)과 완료 포트는 서로 다른 완료 통지 방식이며, 이 글은 완료 포트만 쓴다

반환값: 즉시 완료되면 `0`, 그 외에는 `SOCKET_ERROR`를 반환하며 `WSAGetLastError()`가 `WSA_IO_PENDING`이면 정상적으로 비동기 진행 중이라는 뜻이다. 그 외 에러 코드는 실제 실패이며, 이 경우 완료 통지는 오지 않는다.

공식 문서의 비고 중 이 글의 설계와 관련된 부분은 다음과 같다.

- `WSASend`의 성공(완료)은 상대가 데이터를 받았다는 뜻이 아니다. 전송 계층이 데이터를 받아들였다는 뜻이다.
- 같은 스트림 소켓에 `WSASend`를 동시에 여러 개 걸면 데이터가 섞일 수 있다는 점이 문서에 주의로 적혀 있다. 소켓당 송신을 하나씩만 진행시키고 나머지는 큐에 두는 설계(7.1절)의 이유다.
- 진행 중인 요청마다 `OVERLAPPED`는 별개여야 하고 완료 전까지 유효해야 한다. 버퍼 메모리도 완료 전까지 유효해야 한다. `WSABUF` 배열 자체의 유효 기간은 문서의 파라미터 설명과 비고가 다르게 읽히므로, 이 글은 안전하게 배열도 컨텍스트에 두어 완료 때까지 유지한다.
- 바이트 스트림 소켓에서 수신 완료 바이트가 0이면 상대가 정상 종료(graceful close)했다는 뜻이다(메시지 지향 소켓에서는 `WSAEDISCON`).
- `WSA_OPERATION_ABORTED`는 소켓 닫기나 스레드 종료 등으로 요청이 취소됐을 때 완료 통지로 올 수 있다.

공식 예제 (Microsoft Learn): 두 페이지의 예제는 IOCP가 아니라 이벤트 객체 기반 overlapped I/O이다. `OVERLAPPED.hEvent`에 `WSACreateEvent()`로 만든 이벤트를 넣고 `WSAWaitForMultipleEvents`로 기다린 뒤 `WSAGetOverlappedResult`로 결과를 읽는다. 이 글의 완료 포트 방식과는 완료를 받는 부분만 다르고, `WSA_IO_PENDING`을 정상으로 구분하는 부분과 요청마다 별도의 `OVERLAPPED`를 두는 부분은 같다. 출처와 라이선스는 6.8절에 있다.

`WSASend` 예제(수신 측 클라이언트가 접속하면 4096바이트를 10번 보낸다):

```
#ifndef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif

#include <Windows.h>

#include <winsock2.h>
#include <ws2tcpip.h>
#include <stdio.h>
#include <stdlib.h>

// Need to link with Ws2_32.lib
#pragma comment(lib, "ws2_32.lib")

#define DATA_BUFSIZE 4096
#define SEND_COUNT   10

int __cdecl main()
```

```
{
    WSADATA wsd;

    struct addrinfo *result = NULL;
    struct addrinfo hints;
    WSAOVERLAPPED SendOverlapped;

    SOCKET ListenSocket = INVALID_SOCKET;
    SOCKET AcceptSocket = INVALID_SOCKET;

    WSABUF DataBuf;
    DWORD SendBytes;
    DWORD Flags;

    char buffer[DATA_BUFSIZE];

    int err = 0;
    int rc, i;

    // Load Winsock
    rc = WSAStartup(MAKEWORD(2, 2), &wsd);
    if (rc != 0) {
        printf("Unable to load Winsock: %d\n", rc);
        return 1;
    }

    // Make sure the hints struct is zeroed out
    SecureZeroMemory((PVOID) & hints, sizeof(struct addrinfo));

    // Initialize the hints to obtain the
    // wildcard bind address for IPv4
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_protocol = IPPROTO_TCP;
    hints.ai_flags = AI_PASSIVE;

    rc = getaddrinfo(NULL, "27015", &hints, &result);
    if (rc != 0) {
        printf("getaddrinfo failed with error: %d\n", rc);
        return 1;
    }

    ListenSocket = socket(result->ai_family,
                          result->ai_socktype, result->ai_protocol);
    if (ListenSocket == INVALID_SOCKET) {
        printf("socket failed with error: %d\n", WSAGetLastError());
```

```
        freeaddrinfo(result);
        return 1;
    }

    rc = bind(ListenSocket, result->ai_addr, (int) result->ai_addrlen);
    if (rc == SOCKET_ERROR) {
        printf("bind failed with error: %d\n", WSAGetLastError());
        freeaddrinfo(result);
        closesocket(ListenSocket);
        return 1;
    }

    rc = listen(ListenSocket, 1);
    if (rc == SOCKET_ERROR) {
        printf("listen failed with error: %d\n", WSAGetLastError());
        freeaddrinfo(result);
        closesocket(ListenSocket);
        return 1;
    }
    // Accept an incoming connection request
    AcceptSocket = accept(ListenSocket, NULL, NULL);
    if (AcceptSocket == INVALID_SOCKET) {
        printf("accept failed with error: %d\n", WSAGetLastError());
        freeaddrinfo(result);
        closesocket(ListenSocket);
        return 1;
    }

    printf("Client Accepted...\n");

    // Make sure the SendOverlapped struct is zeroed out
    SecureZeroMemory((PVOID) & SendOverlapped, sizeof (WSAOVERLAPPED));

    // Create an event handle and setup the overlapped structure.
    SendOverlapped.hEvent = WSACreateEvent();
    if (SendOverlapped.hEvent == NULL) {
        printf("WSACreateEvent failed with error: %d\n", WSAGetLastError());
        freeaddrinfo(result);
        closesocket(ListenSocket);
        closesocket(AcceptSocket);
        return 1;
    }

    DataBuf.len = DATA_BUFSIZE;
    DataBuf.buf = buffer;
```

```
    for (i = 0; i < SEND_COUNT; i++) {

        rc = WSASend(AcceptSocket, &DataBuf, 1,
                     &SendBytes, 0, &SendOverlapped, NULL);
        if ((rc == SOCKET_ERROR) &&
            (WSA_IO_PENDING != (err = WSAGetLastError()))) {
            printf("WSASend failed with error: %d\n", err);
            break;
        }

        rc = WSAWaitForMultipleEvents(1, &SendOverlapped.hEvent, TRUE,
INFINITE,
                                      TRUE);
        if (rc == WSA_WAIT_FAILED) {
            printf("WSAWaitForMultipleEvents failed with error: %d\n",
                   WSAGetLastError());
            break;
        }

        rc = WSAGetOverlappedResult(AcceptSocket, &SendOverlapped,
&SendBytes,
                                    FALSE, &Flags);
        if (rc == FALSE) {
            printf("WSASend failed with error: %d\n", WSAGetLastError());
            break;
        }

        printf("Wrote %d bytes\n", SendBytes);

        WSAResetEvent(SendOverlapped.hEvent);

    }

    WSACloseEvent(SendOverlapped.hEvent);
    closesocket(AcceptSocket);
    closesocket(ListenSocket);
    freeaddrinfo(result);

    WSACleanup();

    return 0;
}
```

`WSARecv` 예제(서버 이름을 인자로 받아 접속하고 0바이트가 올 때까지 수신한다):

```
#ifndef UNICODE
#define UNICODE
#endif

#ifndef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif

#include <Windows.h>

#include <winsock2.h>
#include <ws2tcpip.h>
#include <stdio.h>
#include <stdlib.h>

// Need to link with Ws2_32.lib
#pragma comment(lib, "ws2_32.lib")

#pragma warning(disable: 4127)  // Conditional expression is a constant

#define DATA_BUFSIZE 4096

int __cdecl main(int argc, char **argv)
{
    WSADATA wsd;
    struct addrinfo *result = NULL, *ptr = NULL, hints;
    WSAOVERLAPPED RecvOverlapped;
    SOCKET ConnSocket = INVALID_SOCKET;
    WSABUF DataBuf;
    DWORD RecvBytes, Flags;
    char buffer[DATA_BUFSIZE];

    int err = 0;
    int rc;

    if (argc != 2) {
        wprintf(L"usage: %s server-name\n", argv[0]);
        return 1;
    }
    // Load Winsock
    rc = WSAStartup(MAKEWORD(2, 2), &wsd);
    if (rc != 0) {
        wprintf(L"Unable to load Winsock: %d\n", rc);
        return 1;
    }
    // Make sure the hints struct is zeroed out
```

```
    SecureZeroMemory((PVOID) & hints, sizeof (struct addrinfo));

    // Initialize the hints to retrieve the server address for IPv4
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_protocol = IPPROTO_TCP;

    rc = getaddrinfo(argv[1], "27015", &hints, &result);
    if (rc != 0) {
        wprintf(L"getaddrinfo failed with error: %d\n", rc);
        return 1;
    }

    for (ptr = result; ptr != NULL; ptr = ptr->ai_next) {

        ConnSocket = socket(ptr->ai_family, ptr->ai_socktype, ptr-
>ai_protocol);
        if (ConnSocket == INVALID_SOCKET) {
            wprintf(L"socket failed with error: %d\n", WSAGetLastError());
            freeaddrinfo(result);
            return 1;
        }

        rc = connect(ConnSocket, ptr->ai_addr, (int) ptr->ai_addrlen);
        if (rc == SOCKET_ERROR) {

            if (WSAECONNREFUSED == (err = WSAGetLastError())) {
                closesocket(ConnSocket);
                ConnSocket = INVALID_SOCKET;
                continue;
            }
            wprintf(L"connect failed with error: %d\n", err);
            freeaddrinfo(result);
            closesocket(ConnSocket);
            return 1;
        }
        break;
    }
    if (ConnSocket == INVALID_SOCKET) {
        wprintf(L"Unable to establish connection with the server!\n");
        freeaddrinfo(result);
        return 1;
    }

    wprintf(L"Client connected...\n");
```

```
    // Make sure the RecvOverlapped struct is zeroed out
    SecureZeroMemory((PVOID) & RecvOverlapped, sizeof (WSAOVERLAPPED));

    // Create an event handle and setup an overlapped structure.
    RecvOverlapped.hEvent = WSACreateEvent();
    if (RecvOverlapped.hEvent == NULL) {
        wprintf(L"WSACreateEvent failed: %d\n", WSAGetLastError());
        freeaddrinfo(result);
        closesocket(ConnSocket);
        return 1;
    }

    DataBuf.len = DATA_BUFSIZE;
    DataBuf.buf = buffer;

    // Call WSARecv until the peer closes the connection
    // or until an error occurs
    while (1) {

        Flags = 0;
        rc = WSARecv(ConnSocket, &DataBuf, 1, &RecvBytes, &Flags,
&RecvOverlapped, NULL);
        if ((rc == SOCKET_ERROR) && (WSA_IO_PENDING != (err =
WSAGetLastError()))) {
            wprintf(L"WSARecv failed with error: %d\n", err);
            break;
        }

        rc = WSAWaitForMultipleEvents(1, &RecvOverlapped.hEvent, TRUE,
INFINITE, TRUE);
        if (rc == WSA_WAIT_FAILED) {
            wprintf(L"WSAWaitForMultipleEvents failed with error: %d\n",
WSAGetLastError());
            break;
        }

        rc = WSAGetOverlappedResult(ConnSocket, &RecvOverlapped, &RecvBytes,
FALSE, &Flags);
        if (rc == FALSE) {
            wprintf(L"WSARecv operation failed with error: %d\n",
WSAGetLastError());
            break;
        }

        wprintf(L"Read %d bytes\n", RecvBytes);
```

```
        WSAResetEvent(RecvOverlapped.hEvent);

        // If 0 bytes are received, the connection was closed
        if (RecvBytes == 0)
            break;
    }

    WSACloseEvent(RecvOverlapped.hEvent);
    closesocket(ConnSocket);
    freeaddrinfo(result);

    WSACleanup();

    return 0;
}
```

6.6 AcceptEx

`accept()`의 비동기 버전. 클라이언트 연결 수립 자체를 overlapped I/O로 처리해 IOCP 완료 큐로 통지받을 수 있게 한다.

```
BOOL AcceptEx(
    SOCKET       sListenSocket,
    SOCKET       sAcceptSocket,
    PVOID        lpOutputBuffer,
    DWORD        dwReceiveDataLength,
    DWORD        dwLocalAddressLength,
    DWORD        dwRemoteAddressLength,
    LPDWORD      lpdwBytesReceived,
    LPOVERLAPPED lpOverlapped
);
```

파라미터	설명
`sListenSocket`	리슨 중인 소켓
`sAcceptSocket`	미리 `WSASocket(WSA_FLAG_OVERLAPPED)`로 생성만 해둔, bind도 연결도 되지 않은 소켓. 연결이 완료되면 이 소켓이 클라이언트와 통신하는 소켓이 됨
`lpOutputBuffer`	로컬/원격 주소 정보와(옵션으로) 최초 수신 데이터를 함께 받을 버퍼
`dwReceiveDataLength`	`lpOutputBuffer` 중 최초 수신 데이터에 할당할 크기. `0`이면 연결 수립 즉시 완료 통지(데이터 도착까지 기다리지 않음)

파라미터	설명
`dwLocalAddressLength` / `dwRemoteAddressLength`	주소 구조체 저장용 크기. 각각 사용하는 전송 프로토콜의 최대 주소 크기 + 16바이트 이상이어야 함(IPv4는 `sizeof(sockaddr_in) + 16`, IPv6는 `sizeof(sockaddr_in6) + 16`)
`lpdwBytesReceived`	[출력, 동기 완료 시] 수신된 바이트 수
`lpOverlapped`	overlapped 구조체 — IOCP 완료 통지에 사용됨

`lpOutputBuffer`는 반드시 지정해야 하고(수신 데이터를 받지 않아도 주소 정보 저장에 쓰인다), `dwLocalAddressLength` / `dwRemoteAddressLength`는 0일 수 없다. 연결이 수립되기 전에 상대가 끊으면 `WSAECONNRESET`이 보고될 수 있다.

`AcceptEx`는 즉시 성공하면 `TRUE`를, 아니면 `FALSE`를 반환하며 `WSAGetLastError()`가 `ERROR_IO_PENDING`(`WSA_IO_PENDING`과 같은 값 997)이면 진행 중이라는 뜻이다. `lpdwBytesReceived`는 동기 완료일 때만 채워지고, 비동기 완료에서는 완료 통지로 바이트 수를 받는다.

`AcceptEx`는 미리 만들어 둔 소켓을 받으므로, 연결이 들어오는 시점에 소켓을 만드는 비용을 미리 치러 둘 수 있다. 연결 종료 후 그 소켓을 `DisconnectEx(TF_REUSE_SOCKET)`로 되돌려 다시 `AcceptEx`에 넘기는 재사용도 가능하지만(3.6절), 재사용하지 않으면 `AcceptEx`를 걸 때마다 새 소켓과 새 컨텍스트를 만든다. 완료 후에는 `GetAcceptExSockaddrs()`로 로컬/원격 주소를 파싱하고, `setsockopt(SO_UPDATE_ACCEPT_CONTEXT)`로 새 소켓에 리슨 소켓의 속성을 상속시켜야 `getsockname` / `getpeername` 등이 정상 동작한다.

공식 예제 (Microsoft Learn): 확장 함수 포인터를 `WSAIoctl(SIO_GET_EXTENSION_FUNCTION_POINTER)`로 얻고, `AcceptEx`를 한 번 호출한 뒤 수락 소켓을 완료 포트에 연결하는 단일 호출 예제이다.

```
#ifndef UNICODE
#define UNICODE
#endif

#define WIN32_LEAN_AND_MEAN

#include <winsock2.h>
#include <ws2tcpip.h>
#include <mswsock.h>
#include <stdio.h>

// Need to link with Ws2_32.lib
#pragma comment(lib, "Ws2_32.lib")

int main()
```

```
{
    //-----------------------------------------
    // Declare and initialize variables
    WSADATA wsaData;
    int iResult = 0;
    BOOL bRetVal = FALSE;

    HANDLE hCompPort;
    HANDLE hCompPort2;

    LPFN_ACCEPTEX lpfnAcceptEx = NULL;
    GUID GuidAcceptEx = WSAID_ACCEPTEX;
    WSAOVERLAPPED olOverlap;

    SOCKET ListenSocket = INVALID_SOCKET;
    SOCKET AcceptSocket = INVALID_SOCKET;
    sockaddr_in service;
    char lpOutputBuf[1024];
    int outBufLen = 1024;
    DWORD dwBytes;

    hostent *thisHost;
    char *ip;
    u_short port;

    // Initialize Winsock
    iResult = WSAStartup(MAKEWORD(2, 2), &wsaData);
    if (iResult != NO_ERROR) {
        wprintf(L"Error at WSAStartup\n");
        return 1;
    }

    // Create a handle for the completion port
    hCompPort = CreateIoCompletionPort(INVALID_HANDLE_VALUE, NULL, (u_long)
0, 0);
    if (hCompPort == NULL) {
        wprintf(L"CreateIoCompletionPort failed with error: %u\n",
            GetLastError() );
        WSACleanup();
        return 1;
    }

    // Create a listening socket
    ListenSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (ListenSocket == INVALID_SOCKET) {
        wprintf(L"Create of ListenSocket socket failed with error: %u\n",
```

```
            WSAGetLastError() );
        WSACleanup();
        return 1;
    }

    // Associate the listening socket with the completion port
    CreateIoCompletionPort((HANDLE) ListenSocket, hCompPort, (u_long) 0, 0);

    //-----------------------------------------
    // Bind the listening socket to the local IP address
    // and port 27015
    port = 27015;
    thisHost = gethostbyname("");
    ip = inet_ntoa(*(struct in_addr *) *thisHost->h_addr_list);

    service.sin_family = AF_INET;
    service.sin_addr.s_addr = inet_addr(ip);
    service.sin_port = htons(port);

    if (bind(ListenSocket, (SOCKADDR *) & service, sizeof (service)) ==
SOCKET_ERROR) {
        wprintf(L"bind failed with error: %u\n", WSAGetLastError());
        closesocket(ListenSocket);
        WSACleanup();
        return 1;
    }

    //-----------------------------------------
    // Start listening on the listening socket
    iResult = listen(ListenSocket, 100);
    if (iResult == SOCKET_ERROR) {
        wprintf(L"listen failed with error: %u\n", WSAGetLastError());
        closesocket(ListenSocket);
        WSACleanup();
        return 1;
    }

    wprintf(L"Listening on address: %s:%d\n", ip, port);

    // Load the AcceptEx function into memory using WSAIoctl.
    // The WSAIoctl function is an extension of the ioctlsocket()
    // function that can use overlapped I/O. The function's 3rd
    // through 6th parameters are input and output buffers where
    // we pass the pointer to our AcceptEx function. This is used
    // so that we can call the AcceptEx function directly, rather
    // than refer to the Mswsock.lib library.
```

```
    iResult = WSAIoctl(ListenSocket, SIO_GET_EXTENSION_FUNCTION_POINTER,
             &GuidAcceptEx, sizeof (GuidAcceptEx),
             &lpfnAcceptEx, sizeof (lpfnAcceptEx),
             &dwBytes, NULL, NULL);
    if (iResult == SOCKET_ERROR) {
        wprintf(L"WSAIoctl failed with error: %u\n", WSAGetLastError());
        closesocket(ListenSocket);
        WSACleanup();
        return 1;
    }

    // Create an accepting socket
    AcceptSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (AcceptSocket == INVALID_SOCKET) {
        wprintf(L"Create accept socket failed with error: %u\n",
WSAGetLastError());
        closesocket(ListenSocket);
        WSACleanup();
        return 1;
    }

    // Empty our overlapped structure and accept connections.
    memset(&olOverlap, 0, sizeof (olOverlap));

    bRetVal = lpfnAcceptEx(ListenSocket, AcceptSocket, lpOutputBuf,
                outBufLen - ((sizeof (sockaddr_in) + 16) * 2),
                sizeof (sockaddr_in) + 16, sizeof (sockaddr_in) + 16,
                &dwBytes, &olOverlap);
    if (bRetVal == FALSE) {
        wprintf(L"AcceptEx failed with error: %u\n", WSAGetLastError());
        closesocket(AcceptSocket);
        closesocket(ListenSocket);
        WSACleanup();
        return 1;
    }

    // Associate the accept socket with the completion port
    hCompPort2 = CreateIoCompletionPort((HANDLE) AcceptSocket, hCompPort,
(u_long) 0, 0);
    // hCompPort2 should be hCompPort if this succeeds
    if (hCompPort2 == NULL) {
        wprintf(L"CreateIoCompletionPort associate failed with error: %u\n",
            GetLastError() );
        closesocket(AcceptSocket);
        closesocket(ListenSocket);
        WSACleanup();
```

```
        return 1;
    }

    // Continue on to use send, recv, TransmitFile(), etc.,.
    //...

    return 0;
}
```

이 예제를 실제 서버 코드와 비교할 때 주의할 점이 있다. 첫째, `AcceptEx`가 `FALSE`를 반환하면 무조건 실패로 처리해 종료한다. 비동기로 정상 진행될 때도 `FALSE`와 `WSA_IO_PENDING`(997)이 반환되므로, 이 예제는 연결이 즉시 들어와 있지 않으면 실패로 끝난다. 이 예제를 실행하면 `AcceptEx failed with error: 997`로 종료되며, 실제 서버는 997을 정상으로 구분해야 한다. 둘째, 수락 소켓을 `AcceptEx` 호출이 끝난 뒤에 완료 포트에 연결한다. 실제 서버는 `AcceptEx`를 걸기 전에 리슨 소켓을 포트에 연결해 두고, 수락 소켓은 `AcceptEx` 완료를 받은 뒤 포트에 연결하며, `AcceptEx` 호출이 즉시 실패하면 소켓과 컨텍스트를 그 자리에서 회수한다. 셋째, 예제는 완료를 기다리고 처리하는 워커 루프, `SO_UPDATE_ACCEPT_CONTEXT`, 컨텍스트 회수를 생략한 설명용 코드이다. 넷째, `gethostbyname` / `inet_ntoa` / `inet_addr` 같은 구식 API를 쓴다. 신규 코드에는 `getaddrinfo` / `inet_pton`을 쓴다. 다섯째, 수신 길이를 `outBufLen - ((sizeof(sockaddr_in) + 16) * 2)`로 잡아 첫 데이터를 받도록 걸었으므로, 데이터가 올 때까지 완료되지 않는다. 수신 길이를 0으로 하면 연결 수립 즉시 완료된다.

6.7 ConnectEx

`connect()`의 비동기 버전. 클라이언트 역할을 하는 서버(다른 서버로 아웃바운드 연결을 대량으로 맺는 경우)에서 유용하다.

```
BOOL ConnectEx(
    SOCKET         s,
    const sockaddr *name,
    int            namelen,
    PVOID          lpSendBuffer,
    DWORD          dwSendDataLength,
    LPDWORD        lpdwBytesSent,
    LPOVERLAPPED   lpOverlapped
);
```

파라미터	설명
`s`	**미리 `bind()`가 호출되어 있어야 하는** 소켓 (`ConnectEx`의 특이한 제약)
`name` / `namelen`	접속할 대상 주소

파라미터	설명
`lpSendBuffer` / `dwSendDataLength`	연결 수립과 동시에 보낼 초기 데이터(옵션, 없으면 `NULL` / `0`)
`lpdwBytesSent`	[출력, 동기 완료 시] 전송된 바이트 수
`lpOverlapped`	overlapped 구조체

공식 문서의 비고 중 구현에 영향을 주는 부분은 다음과 같다.

- 대상 소켓은 연결 지향 소켓(스트림 등)이어야 하고, `bind`가 먼저 되어 있어야 한다. 주소가 모두 0인 주소로 연결하면 `WSAEADDRNOTAVAIL`이 반환된다. 이미 연결된 소켓에 다시 호출하면 `WSAEISCONN`이다.
- 연결이 성공으로 완료된 뒤 `setsockopt(SO_UPDATE_CONNECT_CONTEXT)`를 호출하기 전에는 `send`, `recv`, `getsockname`, `getpeername`, `shutdown` 등 일부 함수가 정상 동작하지 않는다. 호출 뒤에 이 함수들을 쓴다.
- `lpSendBuffer`로 초기 데이터를 함께 보내는 것은 TCP Fast Open 등 특정 구성에서만 의미가 있고 일반 연결에는 `NULL` / `0`을 쓴다.
- 연결이 닫힌 소켓은 버리고 새로 만든다. 연결 거부, 도달 불가, 타임아웃으로 실패한 경우에는 같은 소켓으로 재시도할 수 있다.
- 연결이 얼마나 오래 진행 중인지는 `getsockopt(SO_CONNECT_TIME)`으로 읽을 수 있다(연결되지 않았으면 `0xFFFFFFFF`, 아래 예제 참고).

공식 예제 (Microsoft Learn): `ConnectEx` 페이지의 예제는 전체 프로그램이 아니라 두 조각이다. 완료 뒤에 소켓 상태를 갱신하는 부분과, `SO_CONNECT_TIME`을 읽는 부분이다.

```
//Need to #include <mswsock.h> for SO_UPDATE_CONNECT_CONTEXT

int iResult = 0;

iResult = setsockopt( s, SOL_SOCKET, SO_UPDATE_CONNECT_CONTEXT, NULL, 0 );
```

```
//Need to #include <mswsock.h> for SO_CONNECT_TIME

int seconds;
int bytes = sizeof(seconds);
int iResult = 0;

iResult = getsockopt( s, SOL_SOCKET, SO_CONNECT_TIME,
                      (char *)&seconds, (PINT)&bytes );
if ( iResult != NO_ERROR ) {
    printf( "getsockopt(SO_CONNECT_TIME) failed with error: %u\n",
        WSAGetLastError() );
}
else {
    if (seconds == 0xFFFFFFFF)
        printf("Connection not established yet\n");
    else
       printf("Connection has been established %ld seconds\n",
           seconds);
}
```

`ConnectEx` 사용 흐름은 다음과 같다. `WSAIoctl` 로 함수 포인터 로딩 → 소켓 생성과 와일드카드 주소로 `bind` → 완료 포트에 연결 → `ConnectEx` 발행(`WSA_IO_PENDING` 은 정상) → 완료 통지에서 성공이면 `SO_UPDATE_CONNECT_CONTEXT` 설정 후 세션으로 승격, 실패면 소켓을 닫음 → 어느 경우든 컨텍스트는 한 번만 회수한다.

`AcceptEx` / `ConnectEx` 는 둘 다 함수 포인터를 `WSAIoctl(SIO_GET_EXTENSION_FUNCTION_POINTER)` 로 직접 얻어와야 하는 Winsock 확장 함수라는 공통점이 있다(RIO의 확장 함수 로딩과 동일한 방식).

`ConnectEx` 는 `TRUE` 로 즉시 성공하거나 `WSA_IO_PENDING` 으로 진행되며, 진행 중이던 연결도 완료 시점에 실패할 수 있다. 따라서 완료 통지에서 성공 여부, 실패 원인, 소켓 상태, 컨텍스트 회수를 함께 처리해야 한다.

I/O를 발행한 스레드가 종료되는 경우에 대해서는 공식 문서(`WSARecv` , `AcceptEx` , `ConnectEx` 의 비고)에 다음 주의가 있다. "스레드가 종료하면 그 스레드가 시작한 모든 I/O는 취소된다. overlapped 소켓에서는 완료 전에 스레드가 닫히면 미완료 비동기 작업이 실패할 수 있다." 이 문장은 Winsock 호출에 공통으로 붙은 주의이고, IOCP와 결합했을 때 어떻게 동작하는지는 문서가 설명하지 않는다. 그래서 특정 동작을 단정하지 않고 다음 원칙으로 대응한다.

- I/O는 완료될 때까지 살아 있는 스레드에서 발행한다. 서버 수명 동안 유지되는 워커나 전용 스레드를 쓰고, 요청마다 만들었다 버리는 스레드에서는 발행하지 않는다.
- 발행 스레드를 먼저 종료하지 않는다. 종료는 4.3절의 순서로 진행 중인 I/O를 모두 회수한 뒤에 한다.
- 이런 이유로 실패한 I/O도 일반 오류와 같은 경로(완료 통지에서 회수)로 처리한다. 발행한 쪽에서 컨텍스트를 따로 해제하지 않는다.

6.8 공식 예제의 출처와 라이선스

6.5~6.7절의 공식 예제는 Microsoft Learn의 해당 함수 페이지(`WSARecv`, `WSASend`, `AcceptEx`, `ConnectEx`)에서 옮긴 것이다. 이 문서화 프로젝트(MicrosoftDocs/sdk-api)의 코드 샘플은 `LICENSE-CODE`에 따라 MIT 라이선스이며 저작권 표기는 "Copyright (c) Microsoft Corporation"이다. 옮겨 적는 과정에서 원문과 공백이나 줄바꿈이 다를 수 있으므로 기준은 항상 원문 페이지이다.

7. 사용법과 사용 예시

7.1 사용 순서

`AcceptEx`로 연결을 받는 IOCP 서버는 아래 순서로 구성한다. 각 단계의 상세는 앞선 절을 참조한다. 7.2절의 예제는 같은 흐름을 일반 `accept()`로 단순화한 것이다.

단계	작업	사용 API
1	Winsock 초기화	`WSAStartup`
2	완료 포트 생성	`CreateIoCompletionPort(INVALID_HANDLE_VALUE, NULL, 0, 0)`
3	리슨 소켓 생성, 확장 함수 로딩, bind, listen, 포트에 연결. 모든 호출의 반환값을 확인하고, 실패하면 이미 만든 자원을 정리한 뒤 종료한다	`WSASocket(WSA_FLAG_OVERLAPPED)`, `WSAIoctl(SIO_GET_EXTENSION_FUNCTION_POINTER)`, `bind`, `listen`, `CreateIoCompletionPort`
4	AcceptEx 다수 선등록. 하나라도 실패하면 기동을 중단하고 정리한다	`AcceptEx`
5	워커 스레드 생성 (프로세서 수의 2배를 출발점으로 하며 필수는 아님, 3.5절). AcceptEx보다 먼저 만들어도 되고, 늦게 시작해도 완료 통지는 큐에 쌓인다	`GetQueuedCompletionStatus` 루프
6	Accept 완료 시: 리슨 컨텍스트 갱신 → 포트 연결 → 세션 생성 → 첫 Recv → AcceptEx 보충	`SO_UPDATE_ACCEPT_CONTEXT`, `CreateIoCompletionPort`, `WSARecv`
7	Recv 완료 시: 데이터 처리 → Recv 재등록	`WSARecv`

단계	작업	사용 API
8	Send 요청 시: 송신 큐에 적재, 진행 중인 Send가 없을 때만 발행	`WSASend`
9	종료: 리슨 소켓 닫기 → AcceptEx/ConnectEx 회수 대기 → 모든 세션 Close 후 소멸 대기 → 워커 수만큼 종료 신호 → 조인 → 회수 불변식 확인	`closesocket`, `PostQueuedCompletionStatus`

설계 원칙은 다음과 같다.

- 소켓당 Recv는 항상 **하나만** 걸어둔다. 완료 후에 다시 건다.
- 소켓당 Send는 **한 번에 하나만** 진행시키고 나머지는 큐에 쌓는다. 동시에 여러 `WSASend`를 걸면 전송 순서와 버퍼 수명 관리가 복잡해진다.
- I/O를 걸 때 컨텍스트가 세션의 `shared_ptr`를 들고, 완료 시 워커가 그것을 꺼내 소유한다(4.2절). 완료 통지가 도착하는 동안 세션은 파괴되지 않는다.
- `CompletionKey`는 식별용으로만 쓰고, 수명 보장은 컨텍스트가 소유한 `shared_ptr`에 맡긴다.
- 소켓을 쓰는 호출과 `closesocket`은 같은 락 안에서 닫힘 플래그를 확인하며 수행한다. `closesocket` 문서는 같은 소켓에 대해 다른 Winsock 호출과 동시에 `closesocket`을 호출하면 안 되고, 닫힌 소켓 핸들 값이 `closesocket` 직후 시스템에 의해 재사용될 수 있다고 밝힌다. `Close()`와 I/O 발행이 경합하면 엉뚱한 연결에 I/O가 걸릴 수 있다.
- 리슨 소켓도 같은 이유로 사용과 종료를 동기화한다. `AcceptEx`와 `SO_UPDATE_ACCEPT_CONTEXT`, 종료 시 `closesocket`을 모두 같은 락 아래에서 수행한다.
- `SendContext`의 소유권: 발행에 성공하면 완료 통지로 넘어가 워커가 `unique_ptr`로 되받고, 발행에 실패하면 발행한 쪽이 해제한다.
- 모든 I/O 종류의 컨텍스트는 4.3절의 규칙으로 회수한다. 종료 신호를 큐에 넣는 것만으로는 안전한 종료가 보장되지 않는다. 신호 뒤에 도착하는 완료는 처리할 워커가 남아 있지 않을 수 있고, 완료 핸들러(재등록, 다음 Recv)가 신호 이후에도 새 I/O를 발행할 수 있기 때문이다. 그래서 새 I/O 발행을 막고(세션 `Close`, 리슨 소켓 종료) 모든 컨텍스트와 세션이 소멸해 진행 중인 I/O가 0이 된 뒤에 신호를 보낸다.

7.2 기본 에코 서버 — 단일 파일 예제

IOCP의 기본 흐름(소켓 등록, 비동기 수신·송신, 완료 처리, 종료)만 한 파일에 담은 가장 단순한 에코 서버다. 연결 수락은 `AcceptEx`가 아니라 일반 `accept()`를 별도 스레드에서 돌리고, 받은 소켓을 IOCP에 등록한다. 그래서 `WSARecv`, `WSASend`, `GetQueuedCompletionStatus`의 동작에 집중해서 읽을 수 있다. 연결 수락까지 비동기로 처리하는 구조는 6.6절의 `AcceptEx`와 7.1절의 순서를 따른다.

완료된 I/O는 IOCP를 거쳐 워커 스레드가 처리한다



IOCP echo flow · 4단계

클라이언트 연결은 accept 스레드가 받아 IOCP에 등록하고, 이후의 수신·송신 완료는 IOCP가 워커 스레드에 전달한다.

설계 원칙은 다음과 같다.

- 비동기 작업마다 독립된 `IoContext`(`OVERLAPPED`와 버퍼 포함)를 쓰고, 완료 통지가 도착하기 전에는 해제하지 않는다. `WSA_IO_PENDING`은 작업이 아직 끝나지 않았다는 뜻이므로 이때 컨텍스트와 버퍼를 해제하면 안 된다.
- 컨텍스트가 세션의 `shared_ptr`를 들고 있어서, 완료 통지가 올 때까지 세션이 파괴되지 않는다.
- 부분 송신은 송신 완료 후 남은 구간을 이어서 다시 송신해서 처리한다. 이전 I/O에 쓴 `OVERLAPPED`를 재사용하기 때문에 다시 걸기 전에 0으로 초기화한다.
- 연결이 끊기면 추가 수신을 걸지 않고, 이미 걸린 작업의 완료 통지를 기다려 회수한다.
- 서버 종료는 새 I/O 등록을 막고, 모든 세션을 닫고, 진행 중인 I/O(`_outstanding`)가 0이 될 때까지 기다린 뒤, 워커 수만큼 종료 신호를 보내고 조인한 다음 IOCP 핸들을 닫는 순서로 진행한다.

C++17 이상으로 빌드하고 `ws2_32.lib`를 링크한다. 포트는 30201이고 Enter를 누르면 종료한다.

```
#define WIN32_LEAN_AND_MEAN

#include <winsock2.h>
#include <windows.h>
#include <ws2tcpip.h>

#include <atomic>
```

```
#include <cstdint>
#include <cstdio>
#include <cstring>
#include <memory>
#include <mutex>
#include <thread>
#include <vector>
#include <string>

#pragma comment(lib, "Ws2_32.lib")

// ============================================================
// 설정
// ============================================================

static constexpr unsigned short SERVER_PORT = 30201;
static constexpr int WORKER_COUNT = 4;
static constexpr int BUFFER_SIZE = 4096;

enum class IoType
{
    Recv,
    Send
};

class CIocpEchoServer;

// ============================================================
// 세션
// ============================================================

struct Session
{
    SOCKET socket = INVALID_SOCKET;
    std::atomic<bool> closing{ false };

    explicit Session(SOCKET s) : socket(s) {}

    void Close()
    {
        if (!closing.exchange(true))
        {
            // 실제 프로젝트에서는 소켓 핸들의 동시 접근 및
            // 등록된 I/O의 완료 수명을 별도로 관리해야 합니다.
            ::shutdown(socket, SD_BOTH);
            ::closesocket(socket);
```

```
        }
    }

    ~Session()
    {
        Close();
    }
};

// ============================================================
// 비동기 작업
//
// OVERLAPPED와 버퍼는 해당 작업의 완료 통지가 올 때까지
// 살아 있어야 합니다.
// ============================================================

struct IoContext
{
    OVERLAPPED overlapped{};
    WSABUF wsaBuf{};

    IoType type = IoType::Recv;
    std::shared_ptr<Session> session;

    char buffer[BUFFER_SIZE]{};
    DWORD transferred = 0;
    DWORD flags = 0;
    DWORD offset = 0;

    explicit IoContext(std::shared_ptr<Session> s, IoType t)
        : type(t), session(std::move(s))
    {
        wsaBuf.buf = buffer;
        wsaBuf.len = BUFFER_SIZE;
    }
};

// ============================================================
// 서버
// ============================================================

class CIocpEchoServer
{
public:
    bool Start()
    {
```

```
        WSADATA wsa{};

        if (::WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
            return false;

        _iocp = ::CreateIoCompletionPort(
            INVALID_HANDLE_VALUE, nullptr, 0, 0);

        if (!_iocp)
        {
            ::WSACleanup();
            return false;
        }

        // IOCP 워커 스레드
        for (int i = 0; i < WORKER_COUNT; ++i)
            _workers.emplace_back(&CIocpEchoServer::Worker, this);

        _listener = ::socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

        if (_listener == INVALID_SOCKET)
        {
            Stop();
            return false;
        }

        sockaddr_in addr{};
        addr.sin_family = AF_INET;
        addr.sin_port = htons(SERVER_PORT);
        addr.sin_addr.s_addr = htonl(INADDR_ANY);

        if (::bind(
                _listener,
                reinterpret_cast<sockaddr*>(&addr),
                sizeof(addr)) == SOCKET_ERROR ||
            ::listen(_listener, SOMAXCONN) == SOCKET_ERROR)
        {
            Stop();
            return false;
        }

        _acceptThread = std::thread(
            &CIocpEchoServer::AcceptLoop, this);

        std::printf("IOCP Echo Server: port %u\n", SERVER_PORT);
        std::printf("Press ENTER to stop.\n");
```

```
        return true;
    }

    void Stop()
    {
        if (_stopping.exchange(true))
            return;

        // AcceptLoop가 accept()에서 빠져나오도록 합니다.
        if (_listener != INVALID_SOCKET)
        {
            ::closesocket(_listener);
            _listener = INVALID_SOCKET;
        }

        if (_acceptThread.joinable())
            _acceptThread.join();

        // 더 이상 새 I/O를 등록하지 않습니다.
        {
            std::lock_guard<std::mutex> lock(_submitMutex);
        }

        std::vector<std::shared_ptr<Session>> sessions;

        {
            std::lock_guard<std::mutex> lock(_sessionsMutex);
            sessions.swap(_sessions);
        }

        for (auto& session : sessions)
            session->Close();

        // 모든 등록 작업의 완료 통지를 소비한 후 워커를 종료합니다.
        while (_outstanding.load() != 0)
            ::Sleep(1);

        for (size_t i = 0; i < _workers.size(); ++i)
            ::PostQueuedCompletionStatus(_iocp, 0, 1, nullptr);

        for (auto& worker : _workers)
        {
            if (worker.joinable())
                worker.join();
        }
```

```
        _workers.clear();

        if (_iocp)
        {
            ::CloseHandle(_iocp);
            _iocp = nullptr;
        }

        ::WSACleanup();
    }

private:
    HANDLE _iocp = nullptr;
    SOCKET _listener = INVALID_SOCKET;

    std::atomic<bool> _stopping{ false };
    std::atomic<long> _outstanding{ 0 };

    std::mutex _submitMutex;
    std::mutex _sessionsMutex;

    std::vector<std::shared_ptr<Session>> _sessions;
    std::vector<std::thread> _workers;
    std::thread _acceptThread;

    // ---------------------------------------------------------
    // 클라이언트 연결 수락
    // ---------------------------------------------------------

    void AcceptLoop()
    {
        while (!_stopping.load())
        {
            SOCKET client = ::accept(_listener, nullptr, nullptr);

            if (client == INVALID_SOCKET)
            {
                if (_stopping.load())
                    break;

                std::printf("accept error: %d\n", WSAGetLastError());
                continue;
            }

            auto session = std::make_shared<Session>(client);
```

```
            HANDLE result = ::CreateIoCompletionPort(
                reinterpret_cast<HANDLE>(client),
                _iocp,
                reinterpret_cast<ULONG_PTR>(session.get()),
                0);

            if (!result)
            {
                session->Close();
                continue;
            }

            {
                std::lock_guard<std::mutex> lock(_sessionsMutex);
                _sessions.push_back(session);
            }

            std::printf("Client connected\n");

            PostRecv(session);
        }
    }

    // ------------------------------------------------------
    // 비동기 수신 등록
    // ------------------------------------------------------

    bool PostRecv(const std::shared_ptr<Session>& session)
    {
        auto* io = new IoContext(session, IoType::Recv);

        std::lock_guard<std::mutex> lock(_submitMutex);

        if (_stopping.load() || session->closing.load())
        {
            delete io;
            return false;
        }

        _outstanding.fetch_add(1);

        int result = ::WSARecv(
            session->socket,
            &io->wsaBuf,
            1,
```

```
            nullptr,
            &io->flags,
            &io->overlapped,
            nullptr);

        if (result == SOCKET_ERROR &&
            ::WSAGetLastError() != WSA_IO_PENDING)
        {
            _outstanding.fetch_sub(1);
            delete io;
            session->Close();
            return false;
        }

        return true;
    }

    // ------------------------------------------------------
    // 비동기 송신 등록
    // ------------------------------------------------------

    bool PostSend(IoContext* io)
    {
        auto session = io->session;

        std::lock_guard<std::mutex> lock(_submitMutex);

        if (_stopping.load() || session->closing.load())
        {
            delete io;
            return false;
        }

        io->type = IoType::Send;

        // 이전 I/O에 쓴 OVERLAPPED를 재사용하므로 0으로 초기화한다.
        std::memset(&io->overlapped, 0, sizeof(OVERLAPPED));

        io->wsaBuf.buf = io->buffer + io->offset;
        io->wsaBuf.len = io->transferred - io->offset;

        _outstanding.fetch_add(1);

        int result = ::WSASend(
            session->socket,
            &io->wsaBuf,
```

```
            1,
            nullptr,
            0,
            &io->overlapped,
            nullptr);

        if (result == SOCKET_ERROR &&
            ::WSAGetLastError() != WSA_IO_PENDING)
        {
            _outstanding.fetch_sub(1);
            delete io;
            session->Close();
            return false;
        }

        return true;
    }

    // ------------------------------------------------------
    // 완료 처리
    // ------------------------------------------------------

    void Worker()
    {
        for (;;)
        {
            DWORD bytes = 0;
            ULONG_PTR key = 0;
            OVERLAPPED* overlapped = nullptr;

            BOOL ok = ::GetQueuedCompletionStatus(
                _iocp,
                &bytes,
                &key,
                &overlapped,
                INFINITE);

            // 종료용 패킷
            if (overlapped == nullptr && key == 1)
                break;

            if (overlapped == nullptr)
                continue;

            auto* io = reinterpret_cast<IoContext*>(overlapped);
            auto session = io->session;
```

```
            if (!ok || bytes == 0)
            {
                session->Close();
                delete io;
                _outstanding.fetch_sub(1);
                continue;
            }

            if (io->type == IoType::Recv)
            {
                // 수신 완료: 받은 데이터를 그대로 송신
                io->transferred = bytes;
                io->offset = 0;

                // PostSend가 io의 소유권을 넘겨받습니다.
                PostSend(io);
            }
            else
            {
                // 부분 송신 처리
                io->offset += bytes;

                if (io->offset < io->transferred)
                {
                    // 남은 데이터 이어서 송신
                    PostSend(io);
                }
                else
                {
                    // 송신 완료 후 다음 수신 시작
                    delete io;

                    if (!session->closing.load() &&
                        !_stopping.load())
                    {
                        PostRecv(session);
                    }
                }
            }

            // 현재 완료된 작업의 outstanding 카운트를 감소시킵니다.
            _outstanding.fetch_sub(1);
        }
    }
};
```

```
// ============================================================
// 진입점
// ============================================================

int main()
{
    CIocpEchoServer server;

    if (!server.Start())
    {
        std::printf("Server start failed.\n");
        return 1;
    }

    std::getchar();

    server.Stop();

    return 0;
}
```

실행과 확인

```
cl /std:c++17 /EHsc /O2 IocpEchoServer.cpp ws2_32.lib
```

(MinGW라면 `g++ -std=c++17 -static IocpEchoServer.cpp -lws2_32`.) 서버를 띄운 뒤 다른 터미널에서 `telnet 127.0.0.1 30201`로 접속해 입력한 문자열이 그대로 돌아오는지 본다. telnet이 없으면 다음 Python 코드를 쓴다.

```
import socket

with socket.create_connection(("127.0.0.1", 30201)) as s:
    s.sendall(b"Hello IOCP Server!")
    print(s.recv(4096).decode())
```

```
Hello IOCP Server!
```

사용한 API

API	역할
`CreateIoCompletionPort`	완료 포트 생성, 소켓을 포트에 등록
`WSARecv`	비동기 수신 요청

API	역할
`WSASend`	비동기 송신 요청
`GetQueuedCompletionStatus`	완료된 I/O 결과 대기와 회수
`PostQueuedCompletionStatus`	워커 스레드에 종료용 완료 패킷 전달
`CloseHandle`	완료 포트 핸들 해제

이 예제의 한계

학습용으로 기본 흐름만 보여 주는 코드이며 운영 서버에 그대로 쓸 수준은 아니다. 다음 문제가 남아 있고, 운영 코드에서는 4장의 원칙으로 해결한다.

- `_outstanding` 카운터만으로는 워커가 새 I/O를 등록하는 시점과 종료 시점의 경합을 완전히 막지 못한다. 종료는 4.3절의 순서(리슨 소켓 닫기 → Accept/Connect 회수 → 모든 세션 Close와 소멸 대기 → 종료 신호)로 진행하고, I/O 발행과 세션 등록을 `Stop()` 과 같은 락으로 직렬화한다.
- `_listener` 를 다른 스레드에서 닫으면서 `accept()` 가 쓰고 있다. 같은 소켓에 대해 다른 Winsock 호출과 동시에 `closesocket` 을 호출하면 안 된다. 리슨 소켓의 사용과 종료를 같은 락 아래에서 수행하거나, accept 스레드가 종료 플래그를 확인하며 빠져나온 뒤 닫는다.
- `Session::Close()` 의 `closesocket` 이 다른 스레드의 `WSARecv` / `WSASend` 와 겹칠 수 있다. 소켓 호출과 `closesocket` 을 세션 락으로 직렬화한다.
- 끊어진 세션을 `_sessions` 에서 제거하지 않아 세션 객체가 종료 시까지 남는다. `weak_ptr` 레지스트리를 쓰고 소멸자에서 제거한다.
- 완료 처리 안에서 새 I/O를 등록하며 `_outstanding` 을 감소시키는 순서가 복잡해서 종료 조건을 정확히 설계하기 어렵다. 컨텍스트 생성·소멸 카운터를 두고 종료 후 불변식을 검사한다(4.3절).
- 수신 데이터를 처리하는 속도가 느린 경우의 백프레셔가 없다. 송신 큐에 상한을 두고 high-water에서 Recv를 멈췄다가 low-water에서 재개한다.

7.3 사용 시 점검 항목

- `WSASocket` 에 `WSA_FLAG_OVERLAPPED` 를 지정했는가 (`socket()` 으로 만든 소켓은 기본으로 overlapped 가능하지만, 명시하는 편이 안전하다).
- 초기화 단계의 모든 호출(`WSAStartup` , `CreateIoCompletionPort` , `WSASocket` , 확장 함수 로딩, `bind` , `listen`)의 반환값을 확인하고, 실패하면 만들어 둔 자원을 정리하고 종료하는가.
- `WSARecv` / `WSASend` 의 `lpCompletionRoutine` 이 `NULL` 이고, overlapped를 쓸 때 바이트 수 인자(`lpNumberOfBytesRecvd` / `lpNumberOfBytesSent`)도 `NULL` 인가.
- `WSA_IO_PENDING` 을 에러로 처리하지 않는가. 반환 `0` (즉시 완료)에도 완료 통지가 큐에 들어오므로 이중으로 처리하지 않는가.
- I/O 발행이 즉시 실패(`WSA_IO_PENDING` 외 에러)하면 완료 통지가 오지 않는다. 이 경우 컨텍스트와 `owner` 를 직접 해제하는가.

- 모든 I/O 종류(Accept, Connect, Recv, Send)에서 컨텍스트 회수 지점이 정확히 하나인가(4.3절 표).
- `OVERLAPPED`와 버퍼가 완료 통지 시점까지 유효한가 (스택 변수 금지).
- 재사용하는 `OVERLAPPED`는 I/O를 다시 걸기 전에 0으로 초기화하는가. 7.2의 부분 송신 재발행이 이 경우다.
- Accept 완료 후 `SO_UPDATE_ACCEPT_CONTEXT`, Connect 완료 후 `SO_UPDATE_CONNECT_CONTEXT`를 설정했는가.
- `closesocket`이 소켓을 쓰는 다른 호출과 같은 락 안에서, `_closed` 확인과 함께 수행되는가. 리슨 소켓도 사용(`AcceptEx`, `SO_UPDATE_ACCEPT_CONTEXT`)과 종료(`closesocket`)가 같은 락 아래에서 동기화되는가.
- 워커 루프가 `GetQueuedCompletionStatus`의 `ov == NULL && FALSE`를 `GetLastError()`로 구분하는가. 유한 타임아웃의 `WAIT_TIMEOUT`은 계속 진행하고, 그 밖의 오류는 공회전하지 않고 종료해야 한다.
- 완료 핸들러가 블로킹 호출(동기 디스크 I/O, Redis, 긴 계산)을 하지 않는가. 필요하면 별도 작업 큐로 넘기고, 워커 수는 프로세서 수의 2배를 출발점으로 측정해서 조정한다.
- 배치 워커가 종료 신호를 만나도 같은 배치의 나머지 항목을 처리하고, 여러 신호를 한꺼번에 소비했다면 되돌려 놓는가.
- 종료 순서가 "신규 accept 중단 → AcceptEx/ConnectEx 회수 대기 → 세션 Close → 세션 소멸 대기 → 종료 신호 → 조인 → 회수 불변식 확인"인가. 종료 신호를 먼저 넣고 나머지를 정리하는 순서가 아닌가.
- I/O(`AcceptEx`, `ConnectEx`, `WSARecv`, `WSASend` 등)를 발행하는 스레드가 완료될 때까지 살아 있는가. 공식 문서는 스레드가 종료하면 그 스레드가 시작한 I/O가 취소되고 overlapped 소켓의 미완료 작업이 실패할 수 있다고 주의한다(6.7절). 초기 `AcceptEx`를 발행한 스레드(`Start` 호출 스레드)도 포함한다.
- 송신 큐에 상한이 있고, 상한에 닿기 전에 Recv를 멈추는 백프레셔가 있는가. high-water에서 Recv를 보류하고 low-water에서 재개하는 방식이 일반적이며, 별도의 상한(예: 1MB)은 최후의 안전장치다.
- 종료 대기용 카운터를 줄이는 쪽이 대기자와 같은 mutex를 거친 뒤 `notify_all`하는가(알림 유실 방지).
- 완료 핸들러가 락을 잡기 전에 `owner`를 지역 변수로 쥐어, 세션이 락이 풀린 뒤에 소멸하는가.
- 세션 레지스트리가 소멸 시 항목을 제거하는가(`weak_ptr`이 남으면 `make_shared` 메모리가 해제되지 않는다).

8. 마무리

IOCP는 "완료를 알려주는" Proactor 모델로, 스레드 수를 커넥션 수와 분리시켜 대규모 동시 접속을 다루는 Windows의 사실상 표준 기술이다. API 자체는 크지 않지만, 실전 안정성은 OVERLAPPED 컨텍스트 설계와 세션 수명 관리(참조 카운팅, close 이후 지연 파괴)에 달려 있다. 이 부분을 제대로 설계해두

면 이후 RIO 같은 더 공격적인 최적화도 같은 원칙 위에서 안전하게 얹을 수 있다.